



Welcome to CSC 365

Internet Programming

Dr. Yifan Zhang

Learning Objectives

1. Understand the difference between imperative and declarative programming.
2. Be able to use declarative functions such as map , filter , reduce, ...

Declarative vs Imperative Programming

- We typically prefer Declarative (what) programming over Imperative (how) programming.
- Declarative array functions include `filter`, `map`, `some`, `every`, and `reduce`.
- These take a callback function as an argument.

Warmup Problem

Given the following array, narrow the elements down such that we only keep elements that are numbers.

```
const things = ["dogs", 1.2, 0, false, {name: "Alice"}, -7]
```

Can you do this imperatively? Use `for (... of ...)`

Can you do this declaratively? Use `filter`

Solution

- Done imperatively ... (how)

```
const newThings = [];  
for(let thing of things) {  
  if (typeof thing === 'number') {  
    newThings.push(thing);  
  }  
}
```

- Done declaratively... (what)

```
const newThings = things.filter((thing) => typeof thing === 'number')
```

Solution

```
1  const things = ["dogs", 1.2, 0, false, {name: "Alice"}, -7]
2
3  let newThings = [];
4  for(let thing of things) {
5    if (typeof thing === 'number') {
6      newThings.push(thing);
7    }
8  }
9
10 function isANumber(something) {
11   return typeof something === 'number'
12 }
13
14 let newThings1 = things.filter(thing => isANumber(thing))
15 let newThings2 = things.filter(isANumber)
16
17 let newThings3 = things.filter((thing) => {
18   return typeof thing === 'number'
19 });
20
21 let newThings4 = things.filter((thing) => typeof thing === 'number')
22
23 console.log(newThings)
24 console.log(newThings1)
25 console.log(newThings2)
26 console.log(newThings3)
27 console.log(newThings4)
```

▼ Array(3) i

0: 1.2

1: 0

2: -7

length: 3

▶ [[Prototype]]: Array(0)

filter

filter performs a function on each element of an array and returns an array of those elements whose function call returned true.

```
const shortNames = ["Bessy", "Rob", "Bartholomew"].filter(name => {  
  if (name.length <= 5) {  
    return true;  
  } else {  
    return false;  
  }  
});  
console.log(shortNames);
```

filter

filter performs a function on each element of an array and returns an array of those elements whose function call returned true.

```
3  const shortNames = ["Bessy", "Rob", "Bartholomew"].filter(name => {  
4    if (name.length <= 5) {  
5      return true;  
6    } else {  
7      return false;  
8    }  
9  });  
10 console.log(shortNames);
```

▼ Array(2) [i](#)

0:	"Bessy"
1:	"Rob"
length:	2
▶ [[Prototype]]:	Array(0)

filter

Using implicit returns, this simplifies to...

```
const shortNames = ["Bessy", "Rob", "Bartholomew"].filter(name => name.length <= 5);  
const longNames = ["Bessy", "Rob", "Bartholomew"].filter(name => name.length > 5);  
console.log(shortNames);  
console.log(longNames);
```

```
▼ Array(2) ⓘ  
  0: "Bessy"  
  1: "Rob"  
  length: 2  
  ► [[Prototype]]: Array(0)
```

```
▼ Array(1) ⓘ  
  0: "Bartholomew"  
  length: 1  
  ► [[Prototype]]: Array(0)
```

map

`map` performs a function on each element of an array and returns an array of the the return of those function calls.

```
const nameLengths = ["Bessy", "Rob", "Bartholomew"].map(n => n.length);  
console.log(nameLengths);
```

```
▼ Array(3) ⓘ  
  0: 5  
  1: 3  
  2: 11  
  length: 3  
  ► [[Prototype]]: Array(0)
```

Chaining Declarative Functions

Of those with short names, how many letters are in their name?

```
["Bessy", "Rob", "Bartholomew"]  
  .filter(name => name.length <= 5)  
  .map(name => name.length);
```

Chaining Declarative Functions

Of those with short names, how many letters are in their name?

```
3  const shortNameLengths = ["Bessy", "Rob", "Bartholomew"]
4  |  .filter(name => name.length <= 5)
5  |  .map(name => name.length);
6  console.log(shortNameLengths);
```

```
▼ Array(2) ⓘ
  0: 5
  1: 3
  length: 2
  ► [[Prototype]]: Array(0)
```

Chaining Declarative Functions

Of those with short names, what are their names and do they have a very short name?

```
[ "Bessy", "Rob", "Bartholomew" ]  
  .filter(n => n.length <= 5)  
  .map(n => {  
    return {  
      name: n,  
      isVeryShort: n.length <= 3  
    }  
  }  
);
```

Chaining Declarative Functions

Of those with short names, what are their names and do they have a very short name?

```
3  const shortAndVeryShort = ["Bessy", "Rob", "Bartholomew"]
4  .filter(n => n.length <= 5)
5  .map(n => {
6    return {
7      name: n,
8      isVeryShort: n.length <= 3
9    }
10 });
11
12 console.log(shortAndVeryShort);
```

▼ Array(2) [i](#)

- ▶ 0: {name: 'Bessy', isVeryShort: false}
- ▶ 1: {name: 'Rob', isVeryShort: true}
- length: 2
- ▶ [[Prototype]]: Array(0)

some(cb) and every(cb)

some(cb) returns true if the callback returns true for some element of the array, false otherwise.

```
["sam", "jacob", "jess"].some(p => p === "jess"); // true!
```

every(cb) returns true if the callback returns true for every element of the array, false otherwise.

```
["sam", "jacob", "jess"].every(p => p === "jess"); // false!
```

reduce(cb, start)

`reduce` takes a 2-parameter callback (previous and current values) and a starting value.

```
[2, 4, -1.1, 7.2].reduce((prev, curr) => prev + curr, 1.2); // 13.3
```

[PythonTutor](#)

` Template Literals

Shorthand for string concatenation;

`${expr}` interpolates a JS expression.

```
21  const student_name = "Patrick Mahomes";
22  const class_name = "cs365";
23  const dt = new Date().toLocaleTimeString();
24
25  console.log("Hello " + student_name + ", welcome to " + class_name + "! It is currently " + dt + ".");
26  console.log(`Hello ${student_name}, welcome to ${class_name}! It is currently ${dt}.`);
```

Hello Patrick Mahomes, welcome to cs365! It is currently 1:39:20 AM.

Hello Patrick Mahomes, welcome to cs365! It is currently 1:39:20 AM.

? Ternary Operator

if, but shorthand!

expr ? if True : if False

```
const age = 17;
let msg;
if (age >= 18) {
  msg = "You are old enough to vote";
} else {
  msg = "You are not old enough"
}
```

```
const age = 17;
const msg = age >= 18 ? "You are old enough to vote" : "You are not old enough";
```

... Spread Operator

Used to create something new with existing data.

We can spread arrays...

```
const cats = ["apricat", "barnaby", "bucky", "colby"];  
const newCats = [...cats, "darcy"];
```

... Spread Operator

... and also objects! These are both shallow copies.

```
const defs = {  
  erf: "a plot of land",  
  popple: "turbulent seas"  
}  
  
const newDefs = {  
  ...defs,  
  futz: "waste of time"  
}
```

Copying

```
let myComplexObj = {name: "Cole", pets: ["pepper", "spud"]};  
  
let refCopy = myComplexObj;           // not really a copy at all!  
let shallowCopy = {...myComplexObj};  // nested attributes not copied  
let deepCopy = JSON.parse(JSON.stringify(myComplexObj)); // full copy!
```

Reference Copy

Shallow Copy

Deep Copy

ICA 6